

Student Name	farjana.S
Register Number	TU-6253210211012
Project Title	Dynamic Portfolio Risk Arbitrage & Value-at-Risk (VaR) Monte Carlo Simulator
Week	Week 5

WEEK-5 – Business Analytics

Task Name

Dynamic Portfolio Risk Arbitrage & Value-at-Risk (VaR) Monte Carlo Simulator

1. Objective

Develop a **multi-asset financial risk simulator** that estimates the possible loss of an investment portfolio under uncertain market conditions.

The system will:

- Analyze **500 financial assets**
- Calculate historical returns
- Build a **500 × 500 covariance matrix**
- Use **Cholesky decomposition** to model correlations
- Generate correlated random market movements
- Simulate future asset prices using **Geometric Brownian Motion (GBM)**
- Calculate the portfolio's **99% Value-at-Risk (VaR)**

2. Input Data

The simulator requires historical price data for **500 assets**.

Example:

Date	Asset 1	Asset 2	Asset 3	...	Asset 500
Day 1	100	250	80	...	150
Day 2	102	248	82	...	152
Day 3	101	253	81	...	149

Required information

- Historical asset prices
 - Portfolio weights
 - Current asset prices
 - Historical time period
 - Risk-free rate, if required
-

3. Calculate Historical Returns

First convert prices into returns.

A commonly used formula is:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

where:

- P_t = current price
- P_{t-1} = previous price
- R_t = return

For example:

If the price changes from ₹100 to ₹105:

$$R = \frac{105 - 100}{100} = 0.05$$

So, the return is **5%**.

4. Covariance Matrix

Calculate the covariance between every pair of assets.

Since there are **500 assets**, the result will be:

500×500 500 \times 500

covariance matrix.

Example:

	Asset 1	Asset 2	Asset 3
Asset 1	0.0025	0.0012	0.0008
Asset 2	0.0012	0.0031	0.0015
Asset 3	0.0008	0.0015	0.0028

The diagonal values represent the **variance** of each asset.

The off-diagonal values represent how two assets move together.

5. Cholesky Decomposition

The covariance matrix is decomposed using **Cholesky decomposition**:

$$\Sigma = LL^T \quad \Sigma = LL^T$$

where:

- Σ = covariance matrix
- LL = lower triangular Cholesky matrix
- L^T = transpose of LL

This is important because it allows the simulator to generate **correlated random variables** instead of treating every asset as completely independent.

6. Generate Random Noise

Generate independent standard normal random variables:

$$Z \sim N(0,1) \quad Z \sim N(0,1)$$

Then transform them using the Cholesky matrix:

$$\epsilon = LZ \quad \epsilon = LZ$$

This produces random movements that have approximately the same correlation structure as the historical assets.

Simple idea

Independent Random Noise

↓

Cholesky Decomposition

↓

Correlated Random Noise

7. Geometric Brownian Motion

Future asset prices are simulated using **Geometric Brownian Motion (GBM)**.

The formula is:

$$S_{t+\Delta t} = S_t \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)\Delta t + \sigma\Delta t Z\right] \quad S_{t+\Delta t} = S_t \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}Z\right]$$

For multiple correlated assets, the random component is replaced by the correlated noise generated using Cholesky decomposition.

Where:

- S_t = current asset price
- μ = expected return
- σ = volatility
- Z = random shock
- Δt = time interval

8. Monte Carlo Simulation

Run the GBM model **thousands of times**.

For example:

500 assets × 10,000 simulations × future time steps

Each simulation represents one possible future market scenario.

Example:

Simulation	Portfolio Future Value
------------	------------------------

1	₹9,82,000
---	-----------

2	₹10,45,000
---	------------

3	₹9,61,000
---	-----------

...	...
-----	-----

10,000	₹8,94,000
--------	-----------

This produces a distribution of possible portfolio values.

9. Portfolio Value

For each simulation:

$$V = \sum_{i=1}^{500} w_i S_i$$

where:

- w_i = portfolio weight

- SiS_i = simulated asset price

The difference between the initial portfolio value and simulated value gives the **profit/loss**.

$$P/L = V_{\text{future}} - V_{\text{initial}} \quad P/L = V_{\{\text{future}\}} - V_{\{\text{initial}\}}$$

10. 99% Value-at-Risk

The **99% VaR** estimates the potential portfolio loss that should not be exceeded in **99% of simulated scenarios**, under the model assumptions.

For example:

Initial portfolio:

₹10,00,000 ₹10,00,000

Suppose the simulated loss distribution gives:

$$VaR_{99\%} = ₹1,25,000 \quad VaR_{\{99\%\}} = ₹1,25,000$$

This means the model estimates that the portfolio could lose about **₹1,25,000 or more in the worst 1% of simulated outcomes** over the specified horizon.

Important

VaR depends on:

- Confidence level
 - Time horizon
 - Historical data
 - Volatility assumptions
 - Correlations
 - Simulation model
-

11. Overall System Flow

Historical Prices – 500 Assets



Data Cleaning



Calculate Returns



Calculate 500×500 Covariance Matrix



Cholesky Decomposition



Generate Independent Random Noise



Create Correlated Noise



Geometric Brownian Motion



Monte Carlo Simulation



Simulated Portfolio Values



Profit / Loss



99% VaR



Risk Analysis Report

12. Expected Output

The final simulator should display:

1. **500-asset covariance matrix**
2. **Cholesky decomposition matrix**
3. **Asset volatility**
4. **Simulated asset prices**
5. **Monte Carlo portfolio-value distribution**

6. **Portfolio profit/loss distribution**
 7. **99% VaR**
 8. **Risk interpretation**
 9. **Optional: Expected Shortfall (CVaR)**
 10. **Charts/dashboard for risk visualization**
-

13. Technologies You Can Use

Programming

- **Python**

Libraries

- **NumPy** – numerical calculations
 - **Pandas** – financial data processing
 - **SciPy** – Cholesky decomposition/statistics
 - **Matplotlib** – visualization
 - **Seaborn** – correlation/covariance visualization
 - **yfinance** – historical market data, if permitted
-

14. Key Learning Outcomes

By completing this task, you demonstrate knowledge of:

- **Financial risk analysis**
- **Portfolio management**
- **Covariance and correlation**
- **Matrix operations**
- **Cholesky decomposition**
- **Monte Carlo simulation**
- **Geometric Brownian Motion**
- **Value-at-Risk**
- **Python-based financial analytics**

